Agentic AI Pipeline for Secure ECG Diagnosis

Integration documentation and results — Data-Poisoning-in-Healthcare

Generated 2026-10-27 from branch **integration**.

Summary

- All eight agents of the architecture are implemented and wired together by one orchestrator, which runs as plain Python or as a LangGraph graph.
- The team's DenseNet + BiLSTM + Attention checkpoint runs inside the pipeline: **91.2%** accuracy on all 15,928 test beats.
- Training-set label audit: **400 of 400** deliberately flipped labels found (precision 0.82).
- Unseen test patients: 12 of 60 clean beats are wrongly flagged (**20%** false positives); 38 of 60 flipped labels and 40 of 60 PGD-attacked beats are flagged.
- **0.67 s per beat** through all agents on a CPU; detectors fit in 54 s.
- Not yet ready for clinical use: the decision thresholds still need calibration, and adversarial signal attacks are mostly attributed to the label (chapter 5).

1. Architecture

A raw ECG beat enters the Validation agent. Valid beats are analysed in parallel by the label poisoning and signal poisoning / noise detectors; the Decision agent fuses their evidence. Clean beats go straight to Diagnosis; attacked beats go through Recovery first. Explainability and the Alert & reporting agent follow. Rejected and quarantined beats skip to the report, which asks for a new recording or expert review.

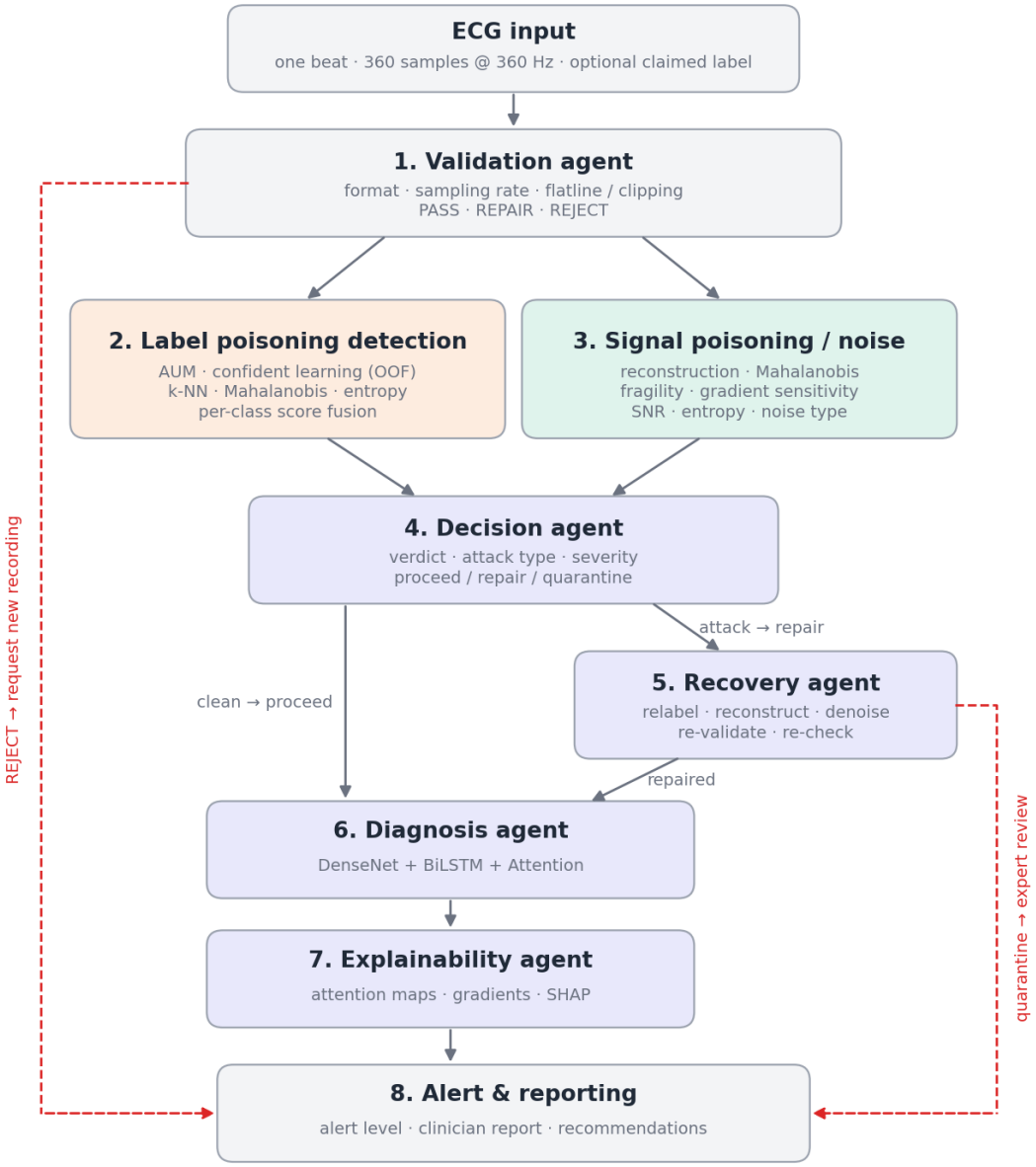


Figure 1 — Pipeline as implemented in orchestrator/pipeline.py.

1.1 Shared contracts

Before integration the branches used three incompatible state formats. All agents now share one state and one set of message types:

Contract	File	Purpose
ECGState	core/ecg_state.py	The dict every pipeline node reads and writes: raw_signal, label, validation, label_poisoning, signal_poisoning, decision, recovery, active_signal, active_label, diagnosis, explainability, report.
Evidence	shared/contracts.py	One detector check: name, channel (label/signal), score in [0, 1], raw value, confidence, metadata.
DetectionReport	shared/contracts.py	Output of each detection agent: evidence, fused score, artifacts.
DecisionRecord	shared/contracts.py	Verdict, confidence, scores, attack type, severity, action, evidence.
RecoveryResult	shared/contracts.py	Recovery action, repaired record, confidence, notes.
ClassifierBackend	models/backend.py	One model interface (predict, embed, gradient, attention) with KerasBackend for the checkpoint and TorchBackend for tests.

1.2 Repository layout

```
agents/
  validation/  label_poisoning/  signal_poisoning/ (+ noise/)  decision/
  recovery/  diagnosis/  explainability/  reporting/
core/          ecg_state.py, calibration.py, signal_filters.py
shared/        contracts.py
models/        backend.py, keras_layers.py, densenet_bilstm.py
orchestrator/  pipeline.py
evaluation/    data.py, synthetic_beats.py, benchmark + metrics scripts
scripts/       run_real_pipeline.py, run_pipeline_demo.py, build_report.py
tests/         end-to-end + real-model tests (agent unit tests live next to each agent)
notebooks/     original Colab notebooks
artifacts/     model + data + results (gitignored)
```

2. The agents

1. Validation agent

Entry point. Verifies that the beat is complete, correctly formatted, sampled at the expected rate and free of obvious corruption, so nothing downstream analyses an unreliable signal.

Files	agents/validation/ (validation_agent.py, format_checks.py, sanity_checks.py, triage.py, repair.py)
Origin	feature/validation-eval branch
Input	Raw beat, shape (360,) or (360, 1); optional metadata (record id, lead, sampling rate).
Output	ValidationResult: status PASS / REPAIR / REJECT, the (possibly repaired) signal, one flag per check, a reason and the list of repair actions.
Configuration	ValidationConfig. Defaults (±5 range, z > 6 clamp) suit raw-amplitude beats. ValidationConfig.zscored_beats() (±12, z > 10) is the preset for the per-beat z-scored ECG_Splits data and is what the pipeline uses.

How it works

- Format checks: shape, numeric dtype, NaN, missing samples, non-finite or out-of-range values.
- Sampling check (added during integration): metadata["fs"] / ["sampling_rate"], if given, must be 360 Hz.
- Signal sanity: flatline (std below 0.01), clipping (≥3 consecutive samples at the rail), abnormal amplitude (per-beat |z| above threshold).
- Triage: structural failure, ≥5% NaN or flatline → REJECT; sparse NaN, clipping or outliers → REPAIR (interpolate NaN, smooth clipped runs, clamp outliers) and re-validate the repaired beat; otherwise PASS.

2. Label poisoning detection agent

Decides whether the beat's annotation can be trusted, i.e. whether the label was flipped by an attacker.

Files	agents/label_poisoning/ (label_poisoning_agent.py, learning_dynamics.py, confident_learning.py, latent_attention.py, score_fusion.py)
Origin	Built during integration from the label-poisoning-agent / poison_detection_agent notebooks (k-NN, Mahalanobis, confidence, entropy, prediction-label disagreement)
Input	Beat, claimed label, record id. Fitted once on the labelled dataset being audited, plus a calibration set of beats from other patients.
Output	DetectionReport (channel "label") with one Evidence per check, the fused label_score in [0, 1] (the attack probability) and the out-of-fold class probabilities used by Recovery to relabel.
Configuration	LabelPoisoningConfig (epochs, folds, k, z_offset, check weights).

How it works

- Learning dynamics (AUM): trains a small classifier on the frozen DenseNet embeddings for 15 epochs and averages each beat's margin p(claimed) − max p(other); mislabeled beats keep a low or negative margin.
- Confident learning: 5-fold out-of-fold probabilities, so no beat is scored by a model that saw its own label; statistic = max p(other) − p(claimed).
- Latent + attention: in the attention-pooled embedding space, the fraction of the 10 nearest neighbours that disagree with the claimed label, Mahalanobis distance to the claimed class centroid relative to the nearest centroid, and −log p(claimed) (label-aware entropy).
- Score fusion with per-class calibration: each statistic becomes a [0, 1] score with robust (median/MAD) statistics per claimed class, then a weighted mean (0.35 / 0.35 / 0.30).
- Dataset audit: suspicious_records() / pipeline.audit_dataset() lists audited training samples above the threshold — the "remove suspicious training samples" option.

3. Signal poisoning / noise detection agent

Detects tampering with the waveform itself and reports signal quality (noise type and severity).

Files	agents/signal_poisoning/ (signal_poisoning_agent.py, reconstruction.py, feature_distance.py, fragility.py) and agents/signal_poisoning/noise/ (spectral noise features + trained classifier)
Origin	Four checks built during integration; noise/ subpackage from the signal-agent branch
Input	Validated beat. Fitted on clean training beats; calibrated on clean beats from other patients.
Output	DetectionReport (channel "signal") with the four Evidence objects the Decision agent fuses, a spectral_noise Evidence, the autoencoder reconstruction and its confidence (for Recovery), and signal_quality: SNR, spectral entropy, noise type, noise severity.
Configuration	SignalPoisoningConfig (perturbation counts and sizes, calibration size, z_offset).

How it works

- Reconstruction check: an undercomplete autoencoder (360-128-32-128-360) trained on clean beats; tampered beats reconstruct badly. Reconstruction confidence = how close the reconstruction is to a fixed point of the autoencoder (i.e. a plausible clean beat).
- Feature-space distance: Mahalanobis distance of the beat's embedding from the clean embedding distribution (Ledoit-Wolf covariance).
- Prediction fragility: total-variation change of the prediction and class-flip rate under 16 small random perturbations.
- Gradient sensitivity: local Lipschitz estimate — the largest input-gradient norm of p(predicted class) at the beat and 7 nearby points.
- Signal quality: wavelet-based SNR estimate, spectral entropy, and the spectral noise agent's baseline-wander / powerline / EMG / morphology features → noise type and severity.
- All four attack statistics are calibrated against clean beats so 0.5 ≈ 3 robust SDs into the tail.

4. Decision agent

Central reasoning step: fuses both detectors' evidence, decides clean vs attacked, names the attack and its severity, and whether recovery is needed.

Files	agents/decision/ (decision_agent.py, fusion.py, config/thresholds.yaml)
Origin	decision-agent / recovery-agent branches; attack type and severity added during integration
Input	label_score + label Evidence; the four signal Evidence objects.
Output	DecisionRecord: verdict (clean, label_poisoned, signal_poisoned, both_poisoned, uncertain), confidence, both scores, attack_type, severity (none/low/moderate/high), recommended action (proceed / repair / quarantine) and all evidence for audit.
Configuration	config/thresholds.yaml (thresholds, uncertain band, fusion strategy, weights).

How it works

- Signal score = weighted mean (default) or noisy-OR of the four signal checks (weights 0.30 / 0.25 / 0.20 / 0.25).
- Thresholds (0.55 / 0.55): label only → repair; signal only → repair; both → quarantine; neither → proceed.
- attack_type from what fired hardest: label_flip, waveform_corruption, out_of_distribution_signal, adversarial_perturbation. Severity from how far past its threshold the triggering score is.

5. Recovery agent

Mitigates a detected attack, or refers the record for expert review when recovery isn't possible.

Files	agents/recovery/ (recovery_agent.py, strategies/label_repair.py, signal_repair.py, denoise.py, quarantine.py, config/repair_thresholds.yaml)
Origin	recovery-agent branch; Butterworth/wavelet denoising added during integration
Input	DecisionRecord, the record, out-of-fold probabilities, autoencoder reconstruction + confidence.
Output	RecoveryResult: action (relabel, reconstruct_signal, denoise_signal, quarantine, reject), the repaired record, repair confidence and notes.
Configuration	config/repair_thresholds.yaml (min confidences, signal_repair_fallback: denoise | quarantine).

How it works

- Label poisoned: relabel to the out-of-fold consensus class if its probability ≥ 0.70, else quarantine.
- Signal poisoned: substitute the autoencoder reconstruction if its confidence ≥ 0.60; otherwise Butterworth 0.5–40 Hz band-pass + db4 wavelet soft-threshold denoising.
- Both poisoned or uncertain: quarantine for manual review.
- Orchestrator safeguard: every repaired beat is re-validated, and a repaired signal must also pass signal-poisoning detection again; otherwise it is quarantined ("referred for expert review").

6. Diagnosis agent

Classifies the validated (or recovered) beat with the DenseNet + BiLSTM + Attention model.

Files	agents/diagnosis/diagnosis_agent.py, models/backend.py, models/keras_layers.py
Origin	feature/validation-eval branch (rewritten onto the shared model backend)
Input	state["active_signal"]: the validated beat or the Recovery agent's repaired beat.
Output	predicted class and label (A, B, N, V), confidence and per-class probabilities.
Configuration	Model path; classes A=0, B=1 (merged LBBB/RBBB), N=2, V=3.

How it works

- Runs the team's DenseNet1D_clean.keras checkpoint (DenseNet blocks → BiLSTM → AttentionLayer → Dense(128) → softmax over 4 classes) through KerasBackend.
- KerasBackend graph-compiles every model call (tf.function), which made single predictions 16× faster than eager calls.

7. Explainability agent

Shows which parts of the beat drove the diagnosis.

Files	agents/explainability/explainability_agent.py
Origin	feature/validation-eval branch (file was truncated mid-line; completed during integration)
Input	The diagnosed beat and its prediction; a background beat (mean of clean training beats).
Output	Attention weights (360), gradient saliency (360), Shapley values for twelve 30-sample (83 ms) segments, and the three most influential regions.
Configuration	Number of SHAP permutations (64 in the pipeline).

How it works

- Attention map: the model's own AttentionLayer softmax weights over BiLSTM timesteps, resampled to 360.
- Gradient saliency: $|\partial p(\text{predicted})/\partial x|$, normalised to [0, 1].
- SHAP: permutation-sampling estimate of segment-level Shapley values against the background beat, computed in one batched forward pass.

8. Alert & reporting agent

Produces the final clinician report.

Files	agents/reporting/reporting_agent.py
Origin	New — written during integration
Input	The complete pipeline state.
Output	Alert level (CRITICAL / WARNING / INFO), validation result, verdict with attack type and severity, per-check evidence, signal quality, recovery action, diagnosis and confidence, most influential regions, recommendations, and a plain-text rendering.
Configuration	Low-confidence and noise-flag thresholds in the module.

How it works

- CRITICAL: validation rejected the beat, or the record was quarantined/rejected — no diagnosis shown.
- WARNING: an attack was detected and repaired — diagnosis shown, marked as from a repaired record.
- INFO: clean beat.
- Recommendations: request a new recording, expert review, confirm a corrected annotation, low diagnostic confidence, abnormal beat class, poor signal quality.

Orchestrator

- orchestrator/pipeline.py: ECGDefensePipeline.from_reference_data() fits both detectors (clean reference beats, the audited dataset, a calibration set from other patients) and builds the pipeline.
- pipeline.run(beat, beat_id, label) runs every node and returns the full ECGState; build_graph() returns the same pipeline as a LangGraph graph with the two detectors as parallel branches.
- Routing: REJECT → report; proceed → diagnosis; repair → recovery → re-validation and re-detection → diagnosis, or quarantine → report.

3. Integration work

What was missing or broken when the branches were reviewed against the architecture, and what was done about it:

Problem found	Fix
No orchestrator; agents lived on separate branches.	Merged all branches (history kept) into one package; wrote orchestrator/pipeline.py (plain Python + LangGraph).
Three incompatible state contracts ("REPAIR" vs "REPAIRED", different keys).	One ECGState in core/, one set of message types in shared/.
Label poisoning existed only as a notebook; no AUM, confident learning or score fusion; Recovery never got the OOF probabilities it needs.	Built the label poisoning agent with all four boxes of the architecture.
Decision agent requires four named signal checks; the signal-agent branch had a different detector.	Built the four checks; the team's spectral noise agent runs alongside as signal-quality evidence.
Diagnosis/Explainability imported a model module that was never committed; explainability_agent.py was truncated mid-line; no SHAP.	Model backend for the Keras checkpoint; explainability completed with attention, gradients and SHAP.
No Alert & reporting agent.	Written.
Noise training script passed arguments the data generator didn't accept.	Added the Gaussian attack mode and arguments to noise_datagen.py.
Validation rejected ~58% of real beats (z-scored R-peaks exceed the ±5 range).	ValidationConfig.zscored_beats() preset — 100% of sampled real beats pass.
Detectors calibrated on training patients flagged 62% of clean test beats.	Calibration on the validation split (other patients): 20%.
Eager Keras calls: 6.2 s per beat, 707 s to fit.	tf.function compilation: 0.67 s per beat, 54 s to fit.
PDF requirements not yet covered (attack type/severity, SNR/entropy, denoising, recommendations, sampling-rate check, removing suspicious training samples).	All added.

4. Results

4.1 Data and model

ECG_Splits CSVs: 360-sample single-lead MIT-BIH beats, z-scored per beat, split by patient (no patient appears in more than one split).

Split	A	B	N	V	Total
train	2,512	10,713	52,409	4,168	69,802
val	5	2,490	9,805	1,982	14,282
test	29	2,123	12,797	979	15,928

4.2 Diagnosis agent on the full test split

Accuracy **91.24%** on 15,928 beats. Class A has only 29 test beats, so its figures are unreliable.

Class	Precision	Recall	F1	Support
A	0.087	0.586	0.151	29
B	0.989	0.461	0.629	2,123
N	0.919	0.986	0.951	12,797
V	0.910	0.934	0.922	979

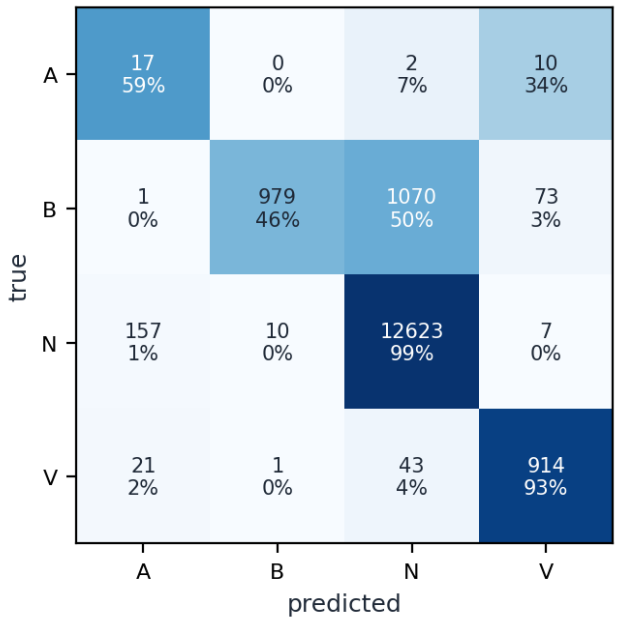


Figure 2 — Diagnosis confusion matrix (test split).

4.3 Validation agent on real beats

Split (2,000 random beats)	PASS with original thresholds	PASS with z-score preset
train	38.6%	100.0%
val	42.4%	100.0%
test	44.1%	100.0%

4.4 End-to-end pipeline on unseen test patients

Setup: detectors fitted on 4,000 clean training beats; label audit on 8,001 training beats of which 400 had their labels deliberately flipped; scores calibrated on 600 validation beats; then 60 unseen test beats per scenario through all eight agents. Decision thresholds: label 0.55, signal 0.55.

Scenario	Flagged	Verdicts	Recovery actions	True class diagnosed
clean	12/60	clean 48, signal poisoned 5, label poisoned 6, both poisoned 1	none 48, quarantine 6, relabel 5, denoise signal 1	45/60
label flipped	38/60	label poisoned 32, clean 22, signal poisoned 1, both poisoned 5	relabel 27, none 22, quarantine 11	43/60
gaussian s=0.01	12/60	clean 48, signal poisoned 5, label poisoned 6, both poisoned 1	none 48, quarantine 6, relabel 5, denoise signal 1	45/60
gaussian s=0.05	13/60	clean 47, signal poisoned 6, label poisoned 6, both poisoned 1	none 47, quarantine 6, relabel 5, denoise signal 2	46/60
PGD eps=0.05	40/60	label poisoned 34, clean 20, both poisoned 4, signal poisoned 2	quarantine 14, none 20, relabel 25, denoise signal 1	21/60

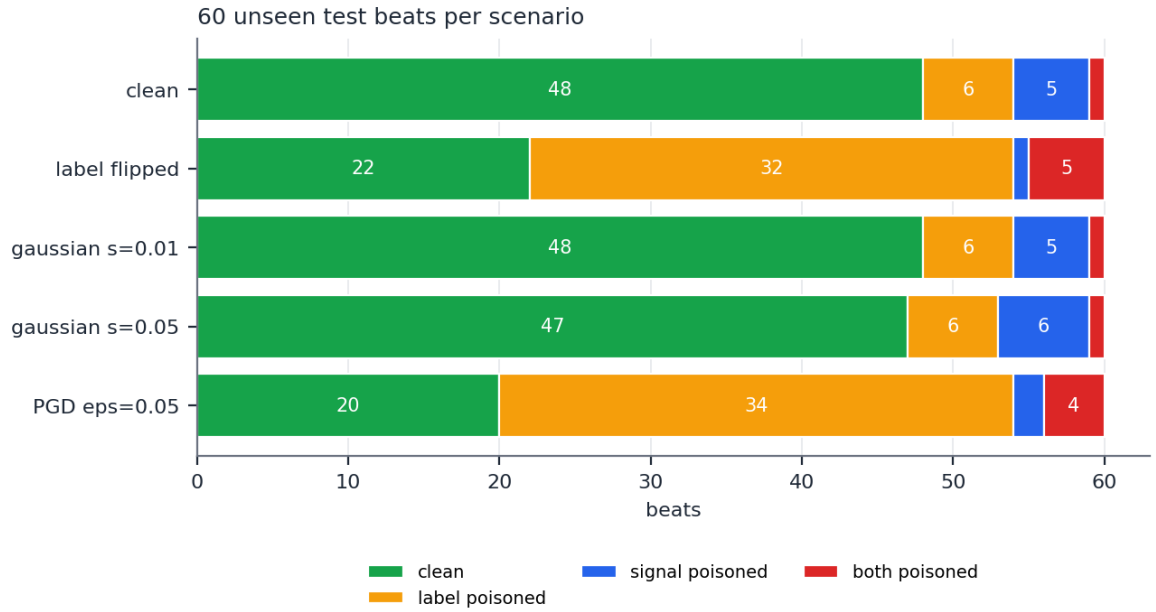


Figure 3 — Decision verdicts per scenario (real model, unseen test patients).

- **Label audit (training set):** recall 1.00, precision 0.82 — the label detector works well on the dataset it audits.
- **Clean beats:** 20% wrongly flagged. These are false positives that send good beats to recovery or review.
- **Gaussian noise σ = 0.01 / 0.05** (the team's poisoning experiment) gives the same verdicts as clean beats: on z-scored beats with R-peaks of 5–8 the noise is imperceptible, and it does not change the diagnosis either.
- **PGD (ε = 0.05):** most attacked beats are flagged, but mainly as label poisoning (chapter 5).
- **True class diagnosed** counts beats whose final diagnosis matches the true class; quarantined beats receive no diagnosis, so they count as misses.

4.5 Score distributions and the fusion choice

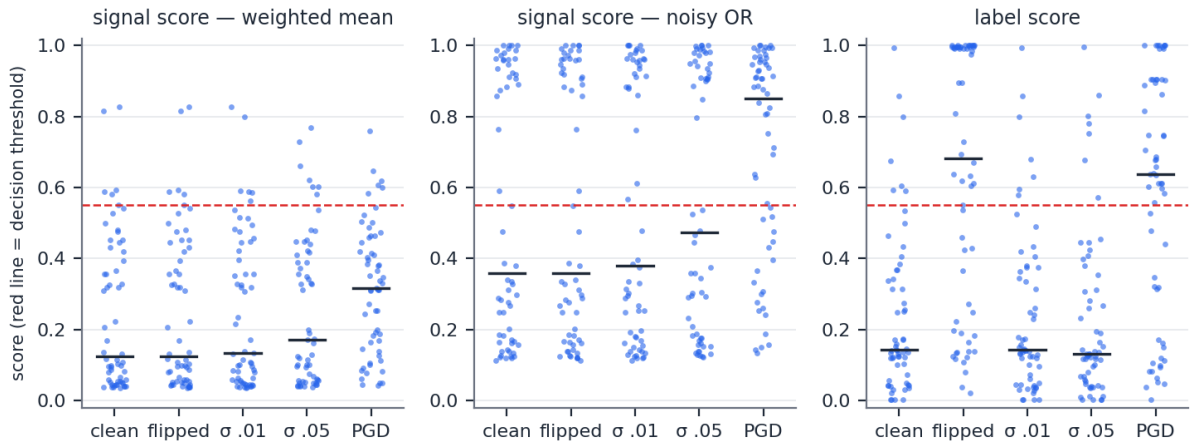


Figure 4 — Per-beat scores by scenario. Black bars are medians, the red line is the decision threshold.

Scenario	Signal flag rate, weighted mean	Signal flag rate, noisy OR
clean	10%	43%
label flipped	10%	43%
gaussian s=0.01	10%	45%
gaussian s=0.05	12%	45%
PGD eps=0.05	10%	67%

The default weighted mean lets three calm checks average down one strong one; noisy OR catches far more PGD beats but also flags many more clean ones. The strategy and the thresholds have to be chosen together on a larger calibration run.

4.6 Effect of the integration fixes

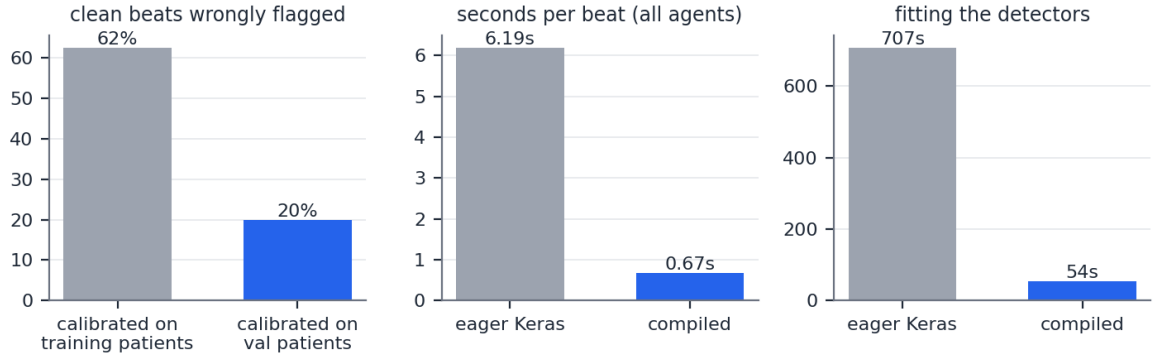


Figure 5 — Before/after the calibration and compilation fixes. "Before" values are development measurements of code that has since been changed.

4.7 Explainability example

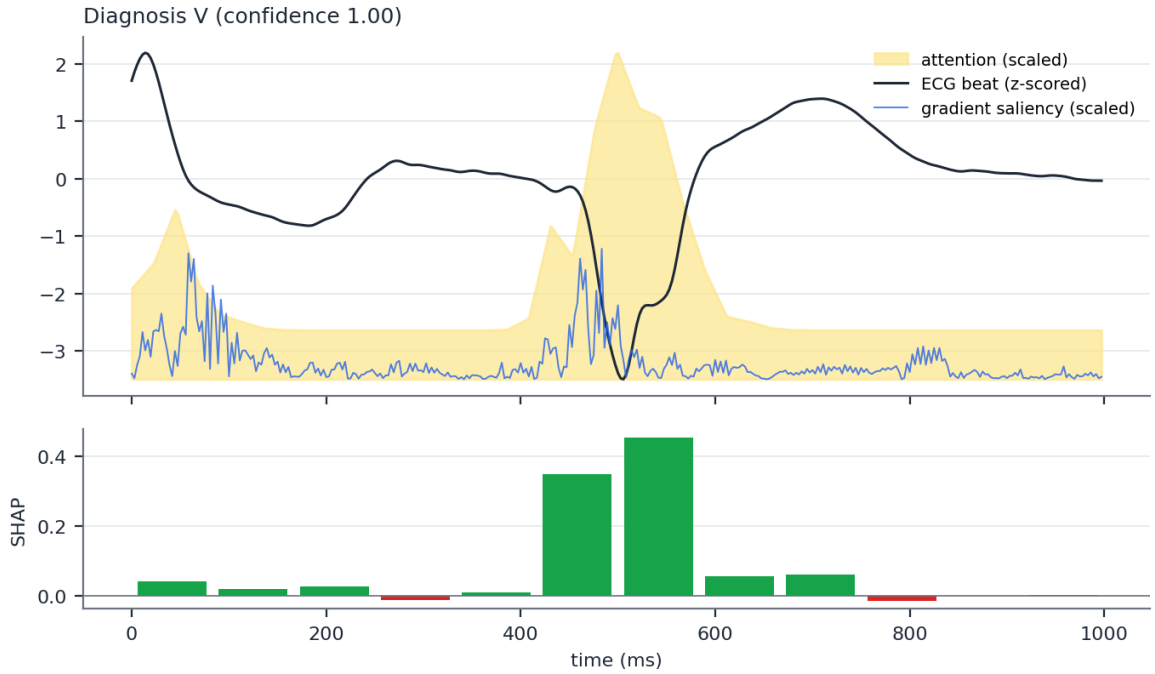


Figure 6 — A V beat from the test split: model attention, gradient saliency, and segment SHAP values (green pushes toward the prediction).

4.8 Example clinician reports

clean

```
ECG beat clean_0 — INFO
  No poisoning detected
  Integrity: verdict=clean (confidence 0.53; label score 0.34, signal score 0.47)
  Signal quality: SNR 50.0 dB, spectral entropy 0.38, noise=morphology (severity 0.57)
  Diagnosis: V (confidence 1.00)
  Most influential regions: samples 150–180 (417–500 ms); samples 180–210 (500–583 ms); samples 60–90 (167–250 ms)
  -> Abnormal beat class predicted (V) — correlate with the full rhythm strip.
```

label flipped

```
ECG beat label flipped_0 — WARNING
  label poisoned detected and repaired (relabel)
  Integrity: verdict=label_poisoned, attack=label_flip, severity=high (confidence 1.00; label score 1.00, signal score 0.47)
  Signal quality: SNR 50.0 dB, spectral entropy 0.38, noise=morphology (severity 0.57)
  Recovery: relabel — relabeled 2 -> 3
  Diagnosis: V (confidence 1.00) [from repaired record]
  Most influential regions: samples 150–180 (417–500 ms); samples 180–210 (500–583 ms); samples 60–90 (167–250 ms)
  -> Annotation corrected 2 -> 3: confirm before using this record for training.
  -> Abnormal beat class predicted (V) — correlate with the full rhythm strip.
```

gaussian s=0.01

```
ECG beat gaussian s=0.01_0 — INFO
  No poisoning detected
  Integrity: verdict=clean (confidence 0.52; label score 0.34, signal score 0.48)
  Signal quality: SNR 34.2 dB, spectral entropy 0.38, noise=morphology (severity 0.57)
  Diagnosis: V (confidence 1.00)
  Most influential regions: samples 150–180 (417–500 ms); samples 180–210 (500–583 ms); samples 60–90 (167–250 ms)
  -> Abnormal beat class predicted (V) — correlate with the full rhythm strip.
```

gaussian s=0.05

```
ECG beat gaussian s=0.05_0 — INFO
  No poisoning detected
  Integrity: verdict=clean (confidence 0.52; label score 0.41, signal score 0.48)
  Signal quality: SNR 23.0 dB, spectral entropy 0.38, noise=morphology (severity 0.57)
  Diagnosis: V (confidence 1.00)
  Most influential regions: samples 180–210 (500–583 ms); samples 150–180 (417–500 ms); samples 60–90 (167–250 ms)
  -> Abnormal beat class predicted (V) — correlate with the full rhythm strip.
```

PGD eps=0.05

```
ECG beat PGD eps=0.05_0 — CRITICAL
  label poisoned detected; record quarantined — relabel confidence 0.68 below min_confidence 0.70
  Integrity: verdict=label_poisoned, attack=label_flip, severity=low (confidence 0.68; label score 0.68, signal score 0.52)
  Signal quality: SNR 29.6 dB, spectral entropy 0.39, noise=morphology (severity 0.56)
  Recovery: quarantine — relabel confidence 0.68 below min_confidence 0.70
  Diagnosis: withheld — manual review required
  -> Expert review required before this recording or its annotation is used clinically or for training.
```

4.9 Synthetic development tests

Before the real model was available, the pipeline was tested on synthetic beats with a small PyTorch DenseNet+BiLSTM+Attention model (tests/test_pipeline_integration.py, scripts/run_pipeline_demo.py). Verdicts on 20 beats per scenario:

Scenario	Verdicts
clean	clean 20
label flipped	label poisoned 19, both poisoned 1
PGD eps=0.05	signal poisoned 12, both poisoned 7, clean 1

Synthetic scenario (100 beats)	weighted mean	noisy OR
clean (unseen)	2%	20%
PGD ε=0.05	22%	67%
PGD ε=0.1	32%	100%
Gaussian σ=0.05	18%	100%

Test suite: 75 tests pass — the agents' own unit tests, end-to-end tests on synthetic beats (including the LangGraph graph), and three tests against the real checkpoint and data.

5. Limitations and next steps

- **False positives on unseen patients (20%).** The thresholds in agents/decision/config/thresholds.yaml are starting values. Sweep them with agents/decision/evaluation/evaluate_fusion.py on a larger validation run.
- **Signal attacks are blamed on the label.** A successful adversarial example makes the classifier disagree with the (correct) label, so for beats outside the audited dataset the label detector fires. 34 of 60 PGD beats were classified as label poisoning, which leads Recovery to relabel or quarantine instead of repairing the signal. A fusion policy under which the signal side also fires turns these into both_poisoned → quarantine.
- **Signal fusion strategy.** Weighted mean vs noisy OR trades missed attacks against false alarms (section 4.5); decide together with the thresholds.
- **Imperceptible noise.** Gaussian σ ≤ 0.05 on z-scored beats is not detected — and does not change diagnoses. A meaningful signal-poisoning evaluation needs attacks that actually affect the classifier (PGD, or larger perturbations).
- **Class mapping.** The model uses 4 classes (A, B, N, V; B = merged LBBB/RBBB) while notebooks/mitbih_preprocessing.ipynb still encodes 5 (A, L, N, R, V). Merge L/R before retraining.
- **Class A** has only 29 test and 5 validation beats, so A-specific calibration and metrics are unreliable.

6. How to reproduce

```
python -m venv venv && venv\Scripts\activate
pip install -r requirements.txt tensorflow reportlab
# put DenseNet1D_clean.keras, train.csv, val.csv, test.csv in artifacts/
python -m pytest -q
python -m scripts.run_real_pipeline --n 60   # -> artifacts/results/real_results.json
python -m scripts.build_report               # -> this PDF
```